

Large Language Model

Alessio Virgillito

March 2026

Indice

1	Preambolo	2
2	Introduzione ai Large Language Model	2
3	La Meccanica degli LLM	2
3.1	I Foundations Models e l'era dei Large Language Models	2
3.2	La Tokenizzazione	3
3.3	Word Embeddings e Spazio Latente	3
3.4	Il ciclo di vita	4
4	Implementazione Pratica, Inference e API	5
4.1	In-Context Learning e Prompt Engineering	6
4.2	RAG (Retrieval-Augmented Generation)	6
4.2.1	Architettura RAG	7
4.2.2	Vantaggi e sfide di RAG	8
4.3	Strumenti e piattaforme per l'implementazione di RAG	9
4.3.1	Database Vettoriali	9
4.4	PEFT (Parameter-Efficient Fine-Tuning)	9
5	Agenti e Sistemi Multi-Agente	10
5.1	Anatomia di un Agente AI	10
5.2	I Sistemi Multi-Agente	10

1 Preambolo

Il seguente documento è stato prodotto con lo scopo di introdurre i concetti basilari sugli LLM e i collegamenti con le discipline Machine and Deep Learning. Lo scopo dei seguenti appunti è puramente didattico e mirato alla costruzione di una Tesi Sperimentale di Primo Livello (Laurea Triennale in Informatica). L'argomento della Tesi mira all'impiego di soluzioni tramite ML (Machine Learning), DL (Deep Learning), LLM (Large Language Model) e AI al mondo giuridico, della sentenza e delle scelte di un giudice artificiale su un caso reale. Per questo motivo gli esempi introdotti sono tutti legati a questo argomento.

2 Introduzione ai Large Language Model

I "Large Language Model", tradotti come "Modelli linguistici di grandi dimensioni" sono una categoria di modelli di **Deep Learning** addestrati su immense quantità di dati che li rendono in grado di comprendere e generare linguaggio naturale e altri tipi di contenuti per eseguire un'ampia gamma di attività. Gli LLM sono basati su un tipo di architettura di **reti neurali** chiamata **trasformatore** che eccelle nella gestione di sequenze di parole e nell'acquisizione di schemi nel testo. In modo estremamente semplificato possiamo pensare gli LLM come gigantesche macchine che prevedono continuamente, tramite statistica, la parola successiva in una sequenza. Imparano gli schemi del testo e generano un linguaggio basato su di essi.

A differenza dei classici motori di ricerca, che semplicemente utilizzano algoritmi di ricerca tradizionale e abbinamenti chiave-valore, il vantaggio sostanziale che portano gli LLM è che le macchine acquisiscono il contesto, sfumature e ragionamenti più profondi. Una volta attribuite funzionalità di **agente** agli LLM, questi possono svolgere, con diversi gradi di autonomia, varie attività che altrimenti verrebbero eseguite dagli essere umani. Oggi per tutti gli LLM sono noti, magari non con nomenclature tecniche, ma nomi come *ChatGPT* - *Copilot* - *Gemini* - *Claude* ecc... sono oramai quotidianità di tutti.

3 La Meccanica degli LLM

3.1 I Foundations Models e l'era dei Large Language Models

Gli LLM rappresentano lo stato dell'arte dell'IA nel campo del Natural Language Processing (NLP). Come definito da IBM e Google, gli LLM non sono modelli isolati, ma appartengono a una categoria più ampia definita **Foundation Models** (Modelli Funzionali): reti neurali di Deep Learning addestrate su quantità di dati non strutturati di scala planetaria. La caratteristica distintiva degli LLM risiede nella loro architettura basata sul **Transformer**, una topologia di rete neurale che eccelle nella gestione di dati sequenziali e nell'acquisizione di pattern semantici a lungo raggio. Da un punto di vista strettamente matematico

e statistico, possiamo definire un LLM come un sofisticato motore probabilistico autoregressivo: il suo compito fondamentale è **calcolare la distribuzione di probabilità del token successivo**, data una sequenza di token in input. Imparando la struttura sintattica e semantica del linguaggio umano durante l'addestramento, l'LLM genera testo contestualmente coerente.

3.2 La Tokenizzazione

Per comprendere come un LLM possa analizzare una complessa sentenza giurisprudenziale, dobbiamo scendere al livello dell'ingegneria dei dati. I computer non leggono "parole", ma elaborano tensori (matrici multidimensionali di numeri). Riguardo il concetto del *Tokenization*, il testo in chiaro non viene immesso direttamente nella rete neurale. Prima di raggiungere il primo strato (Input Layer), il testo grezzo passa attraverso un algoritmo deterministico chiamato **Tokenizer**. Il Tokenizer scompone il testo in unità fondamentali chiamate per l'appunto **Token**. Un token non corrisponde necessariamente a una parola intera: può essere una singola lettera, una sillaba, un suffisso o una parola intera (es. la parola *indiscutibilmente* potrebbe essere divisa in "in", "discutibili", "mente"). L'algoritmo più utilizzato per questa operazione è il **Byte-Pair Encoding (BPE)**, che ottimizza il vocabolario del modello per bilanciare efficienza computazionale e copertura linguistica. A ogni token viene assegnato un ID univoco (un numero intero), che funge da chiave di accesso per il vocabolario del modello.

3.3 Word Embeddings e Spazio Latente

Una volta convertiti in ID numerici, i token vengono trasformati in vettori continui (array di numeri decimali) tramite i **Word Embeddings** (come visto nel capitolo del Deep Learning). Questa proiezione mappa i token in uno spazio vettoriale ad altissima dimensionalità (spesso superiore a 1000 dimensioni). In questo spazio latente, token semanticamente simili (es. "giudice" e "magistrato") assumeranno coordinate vettoriali molto vicine. Questa traduzione matematica è ciò che permette all'LLM di "calcolare" il significato del testo. Rispetto al numero di token invece, un parametro fondamentale di cui tener conto è la **Context Windows**, la cosiddetta Finestra di Contesto. Essa rappresenta il numero massimo di token che l'LLM può "ricordare" ed elaborare simultaneamente in una singola iterazione. Volendo fare un esempio, se un modello ha una Context Window di 8.000 token (circa 6.000 parole) e gli si fornisce in input un fascicolo processuale di 20.000 parole, il modello andrà in errore (*Out of Memory*) o "dimenticherà" (troncherà) la parte iniziale del documento. L'evoluzione tecnologica mira costantemente ad ampliare questo limite (modelli come Gemini di Google ad esempio oggi supportano finestre di contesto di milioni di token).

3.4 Il ciclo di vita

Costruire un LLM moderno non significa semplicemente lanciare un addestramento su un dataset. E' un processo in tre fasi distinte, essenziale per trasformare un generatore di parole in un assistente utile e sicuro.

1. **Fase 1: Pre-Training (Apprendimento Auto-Supervisionato):** Questa è la fase più intensiva e costosa dal punto di vista computazionale. Il modello viene addestrato su trilioni di token (tutto l'internet pubblico) utilizzando l'**apprendimento auto-supervisionato** *Self-Supervised Learning*. Nel capitolo riguardo il Machine Learning abbiamo introdotto il problema delle etichette umane, qui viene aggirato brillantemente. la rete prende una frase ("La legge è uguale per"), nasconde l'ultima parola ("tutti") e tenta di indovinarla. Calcola l'errore (Loss), fa la Retropropagazione (Backpropagation) e aggiorna i suoi miliardi di parametri. Dopo mesi di elaborazione su migliaia di GPU, il "imparato" ha imparato la grammatica, la logica e la conoscenza del modello. Tuttavia, in questa fase, il modello sa solo continuare un testo, non sa rispondere a una domanda. Per fare un esempio di questo concetto, se chiedessimo al modello una domanda come: "Qual è la capitale d'Italia?", potrebbe anche non rispondere "Roma", ma rispondere con: "Qual è la capitale della Francia?", perché sta semplicemente continuando una lista di domande.
2. **Fase 2: Supervised Fine-Tuning (SFT) / Instruction Tuning:** Per trasformare il modello di base in un assistente interattivo, si applica l'addestramento supervisionato tradizionale. Al modello vengono forniti migliaia di esempi strutturati sotto forma di Prompt (La richiesta dell'utente) e *Completamento* (La risposta ideale). In questo modo, l'LLM impara il formato "Domanda → Risposta", diventando in grado di eseguire istruzioni umane, riassumere testi o estrarre capi d'accusa da una sentenza.
3. **Fase 3: Reinforcement Learning from Human Feedback (RLHF):** Anche dopo il SFT, il modello potrebbe ovviamente generare risposte tecnicamente corrette ma eticamente scorrette, tossiche o allucinate (informazioni false). Questo rappresenta e ha rapprensato da sempre uno dei timori principali nell'impiego degli LLM, tanto che spesso si ha paura a fidarsi di ciò che una "macchina" ci dice. Per risolvere questo problema, o quanto meno minimizzare la possibilità di errore, si utilizza il **RLHF**. Dei valutatori umani leggono diverse risposte generate dall'LLM e le classificano dalla migliore alla peggiore. Viene poi addestrata una seconda rete neurale (*Reward Model*) che impara i "gusti" e i valori umani. Infine, questa rete premia o penalizza l'LLM originario, costringendolo ad allinearsi a tre principi fondamentali definiti dalla ricerca sull'IA: essere *utile, onesto e innocuo*.

4 Implementazione Pratica, Inference e API

Poiché il pre-addestramento di un Foundation Model richiede cluster di super-computer e investimenti milionari, lo sviluppo applicativo moderno, soprattutto per scopo accademico, si basa sull'utilizzo di modelli pre-addestrati. L'utilizzo di API o modelli Open Source gioca a favore di chi ovviamente non è un'azienda come Google! Vediamo però prima brevemente cosa sono:

1. **API:** Un API, abbreviazione di "Application Programming Interface", è un insieme di regole, protocolli e strumenti che permette a diversi software di comunicare e scambiare dati tra loro. Agisce come un'interfaccia intermedia, semplificando lo sviluppo di applicazioni consentendo il riutilizzo di codice e funzionalità preesistenti senza dover riscrivere tutto da zero. Il loro funzionamento in breve: Un'applicazione "client" invia una richiesta tramite un indirizzo HTTP a un'altra applicazione "server", che risponde con i dati richiesti, spesso in formato JSON o XML. Esistono diversi tipi di API: Aperte (pubbliche), API Partner (identificazione richiesta) e API interne (private, solo uso aziendale). Per capire le API basti pensare all'integrazione di servizi come Google Maps in un'app di terze parti, o l'uso di X API per recuperare i tweet, o i gateway di pagamento come PayPal.
2. **Modelli Open Weight:** Sono sistemi di IA (specialmente LLM) in cui i "pesi" (come visto quando si è parlato di ML e DL), ossia i parametri che definiscono la conoscenza del modello appresi durante l'addestramento, sono **resi pubblici e scaricabili**. A differenza delle Open Source, non sempre includono il codice di addestramento o i dati, ma permettono di eseguire e personalizzare il modello in locale.

Alcune caratteristiche di questi modelli:

- (a) I pesi (la matrice dei parametri) possono essere scaricati e utilizzati liberamente.
- (b) Esecuzione in locale: consentono di far girare il modello su infrastrutture private, garantendo maggiore privacy e controllo.
- (c) Personalizzazione: Gli sviluppatori possono adattare il modello a compiti specifici utilizzando i propri dati.
- (d) Risparmio: Si elimina la necessità di addestrare un modello da zero (operazione costosa), partendo da una base già performante.

La particolarità di questi modelli è che a differenza dell'Open Source AI, che garantiscono trasparenza totale (codice, dati e pesi), l'open weight fornisce solo i risultati dell'addestramento (i pesi), spesso mantenendo il processo di creazione come una "black-box".

Dunque uno sviluppatore che giustamente non può creare un modello interamente da zero ha due strade principali: *Modelli proprietari tramite API*

oppure si scaricano *modelli aperti* (come Llama 3 di Meta ad esempio) da repository come **Hugging Face**. Poi essi possono essere eseguiti localmente (se si ha una GPU potente) o su piattaforme cloud gratuite/economiche come ad esempio Google Colab. Ma come facciamo a specializzare questi modelli, ad esempio nel dominio giuridico, **senza riaddestrarli** da zero? Esistono varie tecniche che vedremo nel successivo capitolo.

4.1 In-Context Learning e Prompt Engineering

Questo metodo è il più immediato e prevede di sfruttare la *Context Window* del modello. Invece di modificare i pesi della rete neurale, si inseriscono nel Prompt (il testo di input) le istruzioni, il ruolo e alcuni esempi pratici.

- **Zero-Shot Learning:** Si chiede al modello di risolvere il problema senza fornire esempi ("Agisci da giudice civile e valuta questa causa: ...").
- **Few-Shot Learning:** Si inseriscono nel prompt 3 o 4 esempi di sentenze già risolte (ricordiamo le tuple $X \rightarrow y$). L'LLM riconosce il pattern statistico al volo e lo applica al nuovo caso che gli stiamo sottoponendo.

4.2 RAG (Retrieval-Augmented Generation)

Questa è una tecnica più di nicchia. Gli LLM sono addestrati su dati generici e la loro "memoria" si ferma al giorno in cui è finito l'addestramento. Se gli dovessi chiedere ad esempio una sentenza specifica del 2025, inventerà la risposta (Allucinazione). Il **RAG** risolve il problema combinando il motore di ricerca classico con l'LLM:

1. Si convertono tutte le sentenze di *Diritto Pratico* in Word Embedding e si salvano in un "Database Vettoriale".
2. Quando l'utente fa una query (es. "Risolvi questo caso di sinistro stradale"), il sistema cerca nel database i 5 documenti giuridici matematicamente più simili al caso.
3. Questi documenti vengono "iniettati" nel prompt dell'LLM insieme alla domanda originale.
4. L'LLM legge il contesto appena fornito e genera una risposta basata esclusivamente sulle sentenze reali trovate, citando le fonti.

Aggiungiamo adesso una spiegazione dell'Architettura RAG presa dalla fonte: "orq.ai"¹. Il RAG è un approccio all'architettura LLM che integra fonti di conoscenza esterne direttamente nel processo di risposta del modello.

¹Fine-Tuning vs RAG: Key Differences Explained (2025 Guide). Orq.ai Blog: <https://orq.ai/blog/finetuning-vs-rag>

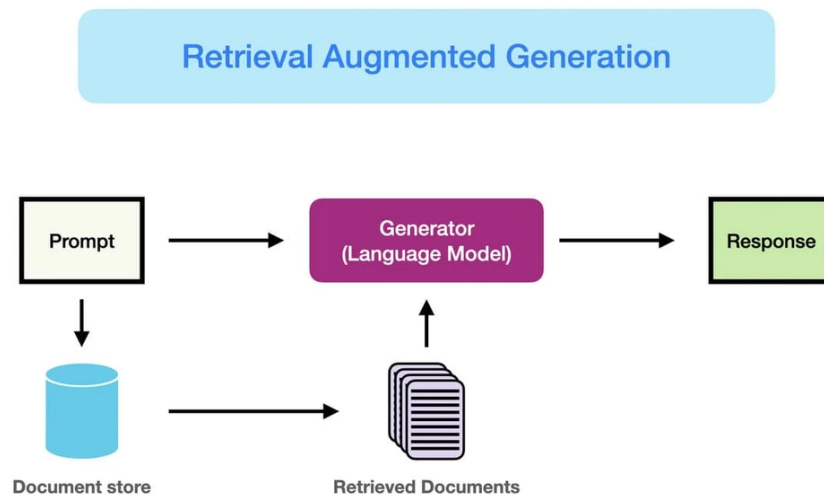


Figura 1: Architettura RAG

Anziché basarsi esclusivamente su ciò che il modello ricorda dal pre-addestramento, i sistemi RAG recuperano informazioni rilevanti e in tempo reale da una **base di conoscenza** connessa e le alimentano al modello per guidarne la **generazione di risposte**. Questo rende RAG particolarmente prezioso in ambiti dinamici in cui i dati cambiano rapidamente o dove la sola messa a punto statica non è sufficiente.

4.2.1 Architettura RAG

Alla base di RAG si trova un'architettura in due parti progettata per l'adattabilità e l'**efficienza dei costi**.

1. **Componente di Recupero:** Questo modulo esegue una **ricerca semantica** su una base di conoscenza strutturata o non strutturata, come un archivio di documenti, un catalogo prodotti o un wiki interno. Utilizza **vettori di embedding** per mappare query e contenuti nello stesso spazio latente, consentendo al sistema di identificare i dati più rilevanti dal punto di vista contestuale.
2. **Componente di generazione:** Una volta recuperati i documenti pertinenti, questi vengono trasmessi a un LLM, che li utilizza per generare una risposta contestualizzata, accurata e fondata.

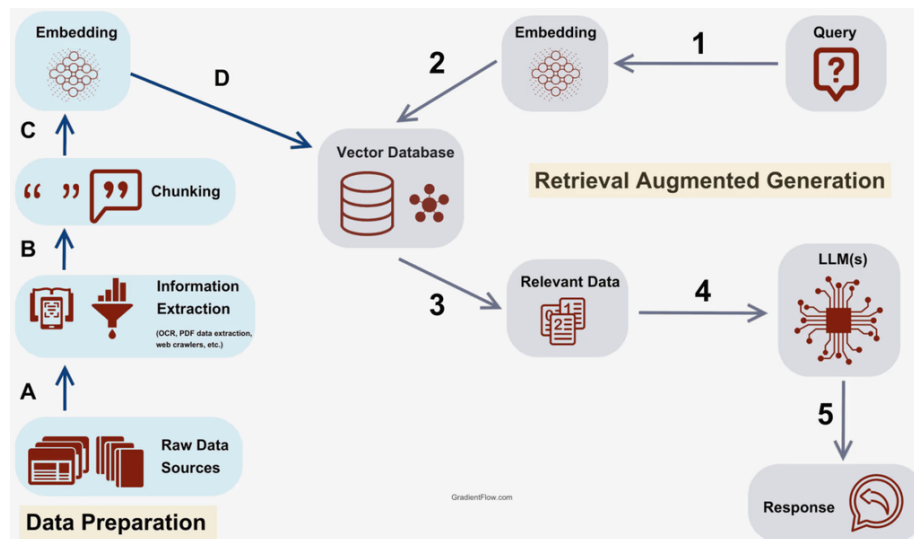


Figura 2: Componente di generazione RAG

Questa configurazione ibrida migliora la **qualità dei dati** e riduce al minimo le allucinazioni ancorando gli output del modello a contenuti reali e verificabili.

4.2.2 Vantaggi e sfide di RAG

RAG introduce una serie di vantaggi che affrontano alcune delle principali limitazioni della **messa a punto dei modelli** tradizionali:

- **Accesso a informazioni dinamiche e in tempo reale:** Recuperando i dati al momento della query, i sistemi RAG rimangono aggiornati senza bisogno di continui cicli di riaddestramento. Ciò è particolarmente utile in ambienti con aggiornamenti frequenti, come specifiche di prodotto, normative o dati finanziari.
- **Riduzione dei costi di ri-addestramento:** rispetto al Fine-Tuning, RAG offre una maggiore **efficienza in termini di costi**, poiché gli aggiornamenti possono spesso essere effettuati modificando le **pipeline di dati** o aggiornando la base di conoscenza anziché riaddestrando il modello stesso.
- **Migliore interpretabilità e affidabilità:** Poiché RAG include citazioni o estratti di contenuto nelle sue risposte, gli utenti possono risalire alla fonte delle risposte, ottenendo **risultati più affidabili** (cosa che in ambito giuridico diventa essenziale), e semplificando le verifiche di conformità nei settori regolamentati.

Sebbene potenti, i sistemi RAG introducono una serie di considerazioni proprie:

- **Dipendenza dalla qualità e dalla pertinenza dei dati:** Un sistema RAG è **valido** solo se i dati e le informazioni che recupera sono di qualità. Dati di scarsa qualità, documenti obsoleti o risultati irrilevanti possono portare a risultati inaccurati, anche se il modello sottostante è solido.
- **Latenza e complessità dell'infrastruttura:** Dovuta soprattutto al recupero di ingenti quantità di dati in architetture non ottimizzate.
- **Sicurezza e conformità:** Poiché RAG spesso attinge a fonti interne, la **sicurezza dei dati** e il controllo degli accessi devono essere gestiti con rigore.

Dunque perché è conveniente l'utilizzo di RAG? Esso è perfetto per scenari dinamici e ricchi di contenuti in cui è essenziale una solida base fattuale. Esempi di utilizzi includono:

- Sintesi e aggregazione di notizie
- **Ricerca e analisi di documenti legali**
- Assistenti alla documentazione tecnica

4.3 Strumenti e piattaforme per l'implementazione di RAG

L'implementazione di RAG richiede più del semplice accesso a un LLM. Per rendere RAG pronto per la produzione, i team devono progettare sistemi scalabili, osservabili e collaborativi in grado di gestire ogni aspetto, dal **recupero dei dati** al monitoraggio **delle prestazioni del modello**.

4.3.1 Database Vettoriali

Al centro di qualsiasi pipeline RAG si trova il **database vettoriale**. Questi database consentono la **ricerca semantica** indicizzando i **vettori di embedding** derivati da documenti e query degli utenti. Quando un utente invia una richiesta, il sistema esegue una ricerca di similarità per recuperare il contesto più rilevante dalla base di conoscenza prima di inserirlo nel LLM. Alcuni strumenti come ad esempio "Pinecone", "Weaviate" e "FAISS" svolgono un ruolo fondamentale nel consentire un **recupero dati** rapido e accurato, soprattutto quando si ha a che fare con corpus di grandi dimensioni che superano la **Context Window** dei modelli tipici. Tuttavia, questi strumenti sono spesso progettati per utenti tecnici, il che rende difficile per i non sviluppatori collaborare o contribuire in modo significativo al flusso di lavoro.

4.4 PEFT (Parameter-Efficient Fine-Tuning)

Se il Prompt Engineering e il RAG non bastano e si vuole modificare il "carattere" del modello open-source, si usa il Fine-Tuning. Poiché aggiornare miliardi di parametri richiederebbe supercomputer, si usano tecniche come LoRA (Low-Rank Adaptation). In sintesi: si "congelano" i miliardi di pesi originali della rete

neurale e si aggiunge un piccolissimo strato di pesi extra (pochi megabyte) che viene addestrato sul dataset di sentenze. Questo si può fare con il sopraccitato Google Colab ad esempio.

5 Agenti e Sistemi Multi-Agente

Un LLM "nudo e crudo" è solo un generatore di testo: gli fai una domanda, ti dà una risposta e si ferma. Per automatizzare processi complessi (come simulare un intero processo giuridico), l'informatica ha sviluppato il concetto di **Agente AI**.

5.1 Anatomia di un Agente AI

Un Agente AI è un programma software (spesso scritto in Python) che utilizza un LLM come suo "cervello" centrale (motore di ragionamento), ma lo dota di tre componenti aggiuntivi:

1. **Persona (System Prompt):** L'identità dell'agente (es. "è un severo giudice del tribunale civile di Catania").
2. **Memoria (Memory):** A differenza di una chiamata API standard che è *stateless*, l'agente mantiene un registro della conversazione e dei passaggi logici precedenti
3. **Strumenti (Tools / Function Calling):** L'LLM viene istruito affinché, invece di rispondere semplicemente a testo, possa generare codice per chiamare funzioni esterne. Un agente può avere uno strumento per fare ricerche su Google, uno per interrogare il Database Vettoriale delle sentenze (RAG), e uno per calcolare la parcella usando una calcolatrice Python.

5.2 I Sistemi Multi-Agente

Se un agente può usare strumenti, cosa succede se il suo "strumento" è un altro agente? Entriamo nel campo dei **Sistemi Multi-Agente (Multi-Agent Systems)**, architetture in cui diverse intelligenze artificiali collaborano (o competono) per risolvere un task complesso. Riprendendo il nostro solito esempio giuridico, si potrebbe implementare una topologia di questo tipo (ESEMPIO):

- **Agente 1 (L'avvocato dell'Attore):** Riceve i fatti iniziali e ha il compito di massimizzare la richiesta di risarcimento, cercando nel database (RAG) le sentenze a lui favorevoli.
- **Agente 2 (L'Avvocato del Convenuto):** Legge la memoria dell'Agente 1 e ha il compito di smontarla, cercando falle logiche e giurisprudenza contraria.

- **Agente 3 (Il Giudice):** Supervisiona il dibattito, analizza le tesi di entrambi gli agenti, verifica se le fonti citate esistono davvero e infine emette la sentenza finale (la famosa variabile y).

Ovviamente, questo esempio sopra citato è un prototipo di come potrebbe essere strutturato un lavoro di tesi giuridico-informatica. Le basi teoriche sugli agenti ci permettono di poter estendere il lavoro in modo personalizzato e dunque costruire una tesi di ricerca differente e mirata. Inoltre, per costruire architetture del genere, gli sviluppatori non scrivono tutto da zero, ma usano framework Python open-source dedicati all'orchestrazione degli LLM. I più famosi e utilizzati sono ad esempio **LangChain**, **LlamaIndex** e framework specifici per agenti come **CrewAI** o **Microsoft AutoGen**.